

CH09 可观测性和日志分析

考试复习笔记——4 大考点全覆盖

来源：开源操作系统课件（43 页） 日期：2026 年 6 月 23 日

4 大考点总览

编号	考点名称	页码范围
考点 1	systemd 服务管理与异常排查	p.10–18
考点 2	日志系统与日志分析	p.19–28
考点 3	日志分析综合案例（网站变慢四步排查）	p.29
考点 4	计划任务：at 与 cron	p.30–42

1 【考点 1】systemd 服务管理与异常排查

1.1 统一对象模型：Linux 视角下的服务

核心公式（Linux 服务统一对象）[p.8]：

Linux 服务 = 进程 + 配置 + 文件 + 网络 + 用户权限 + 资源 + 日志

统一模型的价值：把”故障排查”从零散经验变成结构化问题 [p.8]。

1.1.1 三类服务映射到统一模型 [p.9]

维度	Web 服务	数据库服务	缓存服务 (Redis)
常见软件	Nginx, Java Web, Node, Python	MySQL, PostgreSQL, MariaDB	Redis, Memcached
进程形态	master/worker, 应用进程, 线程池	数据库主进程, 多线程/多进程	Redis 单主进程为主
常见端口	80, 443, 8080	3306, 5432	6379, 11211
配置文件	Nginx 配置, 应用配置	数据库配置	Redis/Memcached 配置
数据文件	静态资源, 上传文件	数据文件, 索引, 事务日志	RDB, AOF 或纯内存
日志	access/error log, 应用日志	error/slow query log, bin-log	Redis log, 持久化日志
运行用户	nginx, www-data, app	mysql, postgres	redis, memcache
关键资源	网络连接, 线程, 静态文件	CPU, 内存, 磁盘 I/O, 锁, 连接数	内存, 网络, CPU, 持久化 I/O

1.1.2 诊断思维：跨服务关联 [p.7]

- Nginx 返回 502 → 根因可能是后端应用进程退出
- 应用接口超时 → 根因可能是数据库慢查询
- **Redis 延迟升高** → 根因可能是磁盘 I/O 被 AOF rewrite 占用
- Web 静态文件 403 → 根因可能是 Linux 目录执行权限或 SELinux
- 服务反复重启 → 根因可能是 systemd 拉起后又因配置错误退出

1.2 systemd 服务单元模型 [p.10–12]

1.2.1 服务单元关键字段 [p.11]

字段	含义
After	描述启动顺序，不等于依赖服务一定健康
User	服务进程以哪个 Linux 用户身份运行
WorkingDirectory	服务进程的当前工作目录
Environment	注入服务运行时环境变量
ExecStart	创建服务进程时执行的命令
Restart	进程异常退出后的重启策略

1.2.2 退出码与信号（核心考点）[p.12]

退出码：程序主动结束时的状态。0 = 正常退出，非 0 = 异常退出。

信号：操作系统通知或控制进程的机制。

信号	信号号	含义与 systemd 显示
SIGTERM	15	请求进程正常终止。code=exited, status=1/FAILURE
SIGKILL	9	强制杀死进程，进程无法捕获。code=killed, status=9/KILL
SIGHUP	1	传统上用于重载配置或终端挂断
SIGSEGV	11	非法内存访问，常见于程序崩溃

重要公式 [p.18]:

$$\text{status} = 128 + \text{信号号}$$

例如：SIGTERM(15) → status = 143；SIGKILL(9) → status = 137

1.2.3 “一切皆文件”与文件描述符 [p.13]

文件描述符是进程访问文件、socket、管道等资源的句柄。常见错误：

- Too many open files —超出文件描述符限制
- accept failed —无法接受新连接
- 无法打开日志文件
- 无法建立新的客户端连接
- 数据库连接失败
- Redis 客户端连接失败

1.3 四类异常及排查方法 [p.14–18]

访问不了（连接断点）→ 访问被拒绝（权限断点）→ 访问变慢（资源断点）→ 服务崩溃（故障断点）

1.3.1 异常一：访问不了——连接层面的断点 [p.15]

检查点	命令	正常表现
服务是否运行	<code>systemctl status nginx</code>	Active: active (running)
端口是否监听	<code>ss -tlnp grep :80</code>	LISTEN 0.0.0.0:80
防火墙是否放行	<code>firewall-cmd --list-all</code>	ports: 80/tcp
依赖服务是否运行	<code>systemctl status mysqld</code>	Active: active (running)
进程是否存在	<code>ps -ef grep nginx</code>	有 nginx 进程

1.3.2 异常二：访问被拒绝——权限层面的断点 [p.16]

检查点	命令	观察内容
文件权限	<code>ls -la /path/to/file</code>	nginx 用户是否有 r/x 权限
SELinux 上下文	<code>ls -laZ /path/to/file</code>	类型是否为 httpd_sys_content_t
SELinux 日志	<code>ausearch -m avc -ts recent</code>	是否有 denied 记录
防火墙规则	<code>iptables -L -n</code>	是否有 REJECT/DROP 规则

1.3.3 异常三：访问变慢——资源层面的断点 [p.17]

资源类型	命令	异常信号
CPU	<code>top</code> 或 <code>htop</code>	%CPU 高, load average > CPU 核心数
内存	<code>free -h</code>	available 低, Swap used > 0
磁盘 I/O	<code>iostat -x 1</code>	%util 接近 100%, await > 10ms
连接数	<code>ss -s</code>	orphaned 或 timewait 过多
文件描述符	<code>ls /proc/\$(pgrep nginx)/fd/ wc -l</code>	接近 <code>ulimit -n</code> 限制

1.3.4 异常四：服务崩溃——故障层面的断点 [p.18] (高频考点)

检查点	命令	观察内容
退出码	<code>systemctl status nginx</code>	status=137 表示被 SIGKILL
退出信号	<code>status = 128 + 信号号</code>	143 = SIGTERM, 137 = SIGKILL
OOM 日志	<code>dmesg grep -i "out of memory"</code>	有 OOM Killer 击杀记录
启动失败	<code>journalctl -u nginx -b</code>	查看最近一次启动日志
重启原因	<code>journalctl -u nginx --since "1 hour ago"</code>	查看最近是否被重启

2 【考点 2】 日志系统与日志分析

2.1 Linux 双日志体系 [p.20]

- 1. `systemd-journald`: `systemd` 管理的系统日志统一由 `journald` 管理，通过 `journalctl` 查看。
- 2. `rsyslog` + `/var/log`: 很多发行版仍把日志写入文本文件，通过直接查看对应文本文件分析。

2.2 journalctl 命令大全 [p.21–22]

2.2.1 基本用法 [p.21]

命令	作用
<code>journalctl -u nginx</code>	查看服务日志
<code>journalctl -u nginx -n 50</code>	查看最近 50 行
<code>journalctl -u nginx --since today</code>	查看今天的日志
<code>journalctl -u nginx -f</code>	实时跟踪日志（类似 <code>tail -f</code> ）
<code>journalctl -u nginx -b</code>	查看上一次成功启动之后的日志
<code>journalctl -u nginx --since "2024-01-15 14:00" --until "..."</code>	按时间范围过滤
<code>journalctl -u nginx -p err</code>	按日志等级过滤（ <code>error</code> 及以上）

2.2.2 分析启动失败的诊断步骤 [p.22]

- 1. 看 `systemctl status` 的 `Active` 状态
- 2. 看 `Main PID` 是否存在
- 3. 看 `Process exited` 后的 `code` 和 `status`
- 4. 看最近日志中第一条 `ERROR` 或 `Exception`
- 5. 看失败发生在哪个阶段：读取配置？绑定端口？连接依赖？权限检查？

典型错误日志示例：

```
petshop.service: Main process exited, code=exited, status=1/FAILURE
java.lang.IllegalStateException: Failed to bind to port 8080
# 直接原因是端口 8080 绑定失败
# 下一步应查看 8080 是否被其他进程占用
```

```
petshop.service: Main process exited, code=killed, status=9/KILL
journalctl -k | grep -i "out of memory"
# 如果内核日志有 OOM 记录，就要考虑内存压力
```

2.3 日志文件格式 [p.23]

以 `/var/log/secure` 为例，日志条目包含四个要素：

1. 事件发生的日期和时间
2. 事件相关的主机名
3. 事件相关进程或函数
4. 事件详细的内容或数据

2.4 rsyslog 日志配置 [p.24–26]

2.4.1 配置语法 [p.24]

服务名称 `[.=!]` 日志等级 记录日志的完整路径及文件名

2.4.2 日志等级（7 级，从低到高） [p.25]

等级号	等级名称	说明
1	info	基本信息的说明
2	notice	必须要通知使用者或系统的信息
3	warning	警告信息，不至于进程停止或崩溃
4	error	错误信息，已导致进程停止或功能不完整
5	crit	比 error 严重，可能导致系统服务中止或模块崩溃
6	alert	该服务或系统的某部分模块已停止工作
7	emerg/panic	最严重，系统已处于崩溃状态

2.4.3 操作符 [p.25]

操作符	含义
.	大于或等于后面等级
.=	只需要所指定等级（精确匹配）
.!	除指定等级外的其他等级（排除）

2.4.4 配置范例 [p.26]

```
news.*;cron.*                /var/log/cronnews
news.=warn;cron.=warn        /var/log/cronnews.warn
*.info;mail.none;news.none;authpriv.none;cron.none /var/log/messages
authpriv.*                   /var/log/secure
mail.*                        -/var/log/maillog
cron.*                        /var/log/cron
*.emerg                       *
```

2.5 Nginx 访问日志 awk 分析 [p.27] （高频考点）

Nginx 日志格式：

```
192.168.1.100 - - [15/Jan/2024:10:30:45 +0800] "GET /index.html HTTP/1.1"
200 612 "https://example.com/" "Mozilla/5.0"
```

常用分析命令速查：

分析目标	命令
统计总请求数（PV）	<code>wc -l /var/log/nginx/access.log</code>
统计独立 IP 数（UV）	<code>awk '{print \$1}' access.log sort uniq wc -l</code>
统计各状态码数量	<code>awk '{print \$9}' access.log sort uniq -c sort -rn</code>
统计 4xx/5xx 错误数	<code>awk '{print \$9}' access.log grep -E "^4 ^5" wc -l</code>
访问最频繁 URL Top 10	<code>awk '{print \$7}' access.log sort uniq -c sort -rn head -10</code>
访问量最大 IP Top 10	<code>awk '{print \$1}' access.log sort uniq -c sort -rn head -10</code>
按时间段统计请求数	<code>awk '{print \$4}' access.log cut -d: -f2 sort uniq -c</code>

3 【考点 3】 日志分析综合案例：网站变慢排查四步法

这是本课最重要的综合应用题，必须完整掌握四步排查流程。

场景：用户报告网站变慢，疑似有故障。需要从日志入手定位根因 [p.29]。

3.1 步骤 1：按时间定位（找异常时间点）

目的：缩小排查范围，找出故障开始的时间窗口。

```
journalctl --since "14:00" --until "14:30" | grep -i error
```

3.2 步骤 2：按服务定位（哪个服务出问题）

目的：确定是哪个服务（Nginx/MySQL/Redis）出现了异常。

```
systemctl status nginx mysqld redis  
journalctl -u nginx mysqld redis --since "14:00" -p err
```

说明：-p err 只显示 error 及以上等级的日志，快速过滤噪音。

3.3 步骤 3：按请求链路定位（哪个环节延迟）

目的：在服务链路中确定延迟发生的具体环节。

```
# 检查 Nginx access log, 找慢请求（假设响应时间 >5 秒的为慢请求）  
awk '$NF > 5 {print}' /var/log/nginx/access.log
```

```
# 检查应用日志，找处理时间长的记录  
grep -i "timeout\\|slow" /var/log/myapp/application.log
```

3.4 步骤 4：按错误关键词定位（根因）

目的：从错误日志中定位最终根因。

```
# 提取应用错误日志（含上下文）  
grep -B2 -A5 "Exception\\|ERROR" /var/log/myapp/application.log | head -50  
  
# 检查数据库慢查询  
grep "Query_time" /var/log/mysql/slow.log | awk '$2 > 1 {print}' | head -10
```

排查四步法记忆口诀：时间 → 服务 → 链路 → 关键词

1. 时间: `journalctl --since ... --until ... | grep -i error`
2. 服务: `systemctl status nginx mysqld redis + journalctl -u ... -p err`
3. 链路: `awk '$NF > 5' Nginx 慢请求 + grep timeout|slow 应用慢日志`
4. 关键词: `grep -B2 -A5 "Exception|ERROR" 应用错误 + grep "Query_time" 慢查询`

Redis 在本案例中的角色：作为缓存服务被检查（步骤 2 中 `systemctl status redis`），如果 Redis 崩溃或连接失败，会导致请求绕过缓存直接打到数据库，引起网站变慢。

4 【考点 4】计划任务：at 与 cron

4.1 计划任务分类 [p.31]

类型	依赖服务	特点
例行性任务 (cron)	crond (默认开启, 常驻内存)	按设定周期自动循环多次执行
单次计划任务 (at)	atd (需确认可用)	按设定的某一时间只执行一次

4.2 计划任务常用场景 [p.32]

1. 日志轮替 (log rotate): 定期备份老日志, 提高日志管理性能
2. 日志分析 (log watch): 定期检测重要日志文件的改动并分析
3. 重建文件索引 (locate): 更新 updatedb 数据库
4. 删除临时文件 (tmpwatch): 定期清除临时文件, 保证磁盘空间
5. Web 分析: 定期对网站日志进行分析

4.3 at 单次计划任务 [p.33–36]

4.3.1 atd 服务管理 [p.33]

```
systemctl status atd      # 查看 at 服务状态
systemctl restart atd     # 重启 at 服务
systemctl enable atd      # 设置开机自动启动 atd 服务
```

4.3.2 at 运行原理 [p.34]

- 每个 at 任务在 /var/spool/at/ 目录内对应一个文件, 保存脚本和时间
- atd 进程通过读取该文件完成 at 任务
- 权限控制: 白名单 /etc/at.allow 优先级高于黑名单 /etc/at.deny
- 两个文件均不存在时, 只有 root 有权设置 at 任务

4.3.3 at 命令语法 [p.35]

```
at [-mldv] TIME
at -c 作业编号
```

选项	说明
-m	当 at 工作完成后, 以 email 通知用户
-l	相当于 atq, 列出所有 at 任务
-d	相当于 atrm, 取消一个 at 任务
-v	用更详细的时间参数列出 at 任务的时间
-c	查看指定编号的 at 任务内容

TIME 参数：HH:MM、HH:MM YYYY-MM-DD 等 Linux 兼容格式。

4.3.4 at 相关命令

命令	作用
<code>at</code>	创建 at 任务，Ctrl+D 退出设置
<code>atq</code>	列出系统中的 at 任务
<code>atrm</code>	删除指定 at 任务

4.4 cron 例行性计划任务 [p.37–42]

4.4.1 crond 服务 [p.37]

- crond 服务默认开启，常驻内存
- 两种设置方式：
 1. 用户计划任务：通过 `crontab` 命令，结合 `/var/spool/cron` 目录和 `/etc/cron.allow`、`/etc/cron.deny` 文件
 2. 系统计划任务：通过编辑 `/etc/crontab` 配置文件

4.4.2 crontab 命令 [p.38]

`crontab [-elr] [-u 用户名]`

选项	说明
<code>-e</code>	进入 cron 任务编辑模式（不可省略）
<code>-l</code>	查看已设置的计划任务
<code>-r</code>	删除计划任务
<code>-u</code>	给其他用户设置计划任务（root 专用）

4.4.3 crontab 时间表示法（核心考点） [p.39]

分钟	小时	日期	月份	周
0–59	0–23	1–31	1–12	0–7（0 和 7 都表示周日）

特殊字符	含义	示例
*	代表取任意值	0 12 * * * → 每天 12:00
,	代表分隔时段	0 3,6 * * * → 每天 3:00 和 6:00
-	代表一段时间内	20 8-12 * * * → 8:00–12:00 之间每小时 20 分
/n	每隔多久单位时间	*/5 * * * * → 每 5 分钟

4.4.4 crontab 使用范例 [p.40]

5月1日23:59发邮件（女朋友生日前一天提醒）

```
59 23 1 5 * mail yoyo < /home/test/lover.txt
```

每五分钟执行一次脚本

```
* /5 * * * * /home/test/test.sh
```

每星期五下午4:30提醒周六约会

```
30 16 * * 5 mail friend@***.com < /home/test/friend.txt
```

本章知识结构总览

考点	页码	核心命令/知识点
考点 1 systemd 异常排查	p.10-18	<code>systemctl status</code> , <code>ss -tlnp</code> , <code>firewall-cmd</code> , <code>ps -ef</code> , <code>top</code> , <code>free -h</code> , <code>iostat</code> , <code>dmesg grep OOM</code> , status = 128 + 信号号, 四类异常断点模型
考点 2 日志系统	p.19-28	<code>journalctl</code> 全套参数, 7 级日志等级, rsyslog.conf 配置语法, <code>awk</code> Nginx 日志分析
考点 3 综合案例	p.29	四步排查法: 时间 → 服务 → 链路 → 关键词 跨 nginx + redis + mysql 联合排查
考点 4 计划任务	p.30-42	<code>at/atq/atrm</code> , <code>crontab -e/-l/-r</code> , crontab 五行时间格式, */,- 特殊字符

开源操作系统 · CH09 可观测性和日志分析 · 考点复习笔记 · 共 4 大考点 · 43 页课件全覆盖